

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт № 8 «Компьютерные науки и прикладная математика»

ОТЧЕТ
по лабораторным работам
по дисциплине «Информационный поиск»

Тема:
Разработка компонентов поисковой системы

Выполнил(а): _____
Студент группы: _____

Преподаватели:

Дата сдачи: Оценка:

Подпись преподавателя:

Москва, 2025

Содержание

1	Постановка задачи и ограничения	3
1.1	Общая цель	3
1.2	Ограничения реализации	3
2	Лабораторная 1. Добыча корпуса документов	3
2.1	Источник данных и тематика	3
2.2	Формат сырых документов	3
2.3	Выделение текста	4
2.4	Журнал выполнения и типовые проблемы	4
2.5	Пример выполнения программы	4
3	Лабораторная 2. Поисковый робот	4
3.1	Требования	4
3.2	Метод решения	5
3.3	Схема хранения документов	5
3.4	Ключевые фрагменты реализации	5
3.5	Экспериментальная проверка работы поискового робота	6
4	Токенизация	6
4.1	Метод токенизации	7
5	Лемматизация	8
5.1	Метод лемматизации	8
5.2	Сравнение частот до и после лемматизации	8
5.3	Результаты после лемматизации	9
5.4	Анализ влияния на качество поиска	9
5.5	Закон Ципфа	9
5.5.1	Распределение токенов без лемматизации	10
5.5.2	Распределение после лемматизации	10
5.5.3	Сравнение и выводы	11
6	Токенизация и нормализация текста	12
6.1	Цель и общая схема	12
6.2	Входные и выходные данные	12
6.3	Настройки токенизации	12
6.4	Декодирование UTF-8 и представление символов	12
6.5	Нормализация символов	13
6.6	Правила формирования токенов	13
6.7	Позиции токенов	13
6.8	Ограничения по длине	13
6.9	Результат токенизации	14
7	Булев индекс	14
7.1	Токенизация и нормализация	14
7.2	Бинарный формат хранения	14
7.2.1	direct.bin (прямой индекс)	14
7.2.2	vocab.bin (словарь)	14
7.2.3	postings.bin (обратный индекс)	15
7.3	Метод сортировки	15

8	Булев поиск и ранжирование (ЛР4 + ЛР8)	15
8.1	Синтаксис запросов и парсер	15
8.2	Модель выполнения запроса	15
8.3	TF-IDF ранжирование	16
8.4	Примеры запросов (удачные и неудачные)	16
8.5	Тестирование корректности (как обосновано)	16
9	Координатное сжатие (ЛР6)	16
9.1	Выбранный метод сжатия и причины выбора	16
9.2	Побитовая схема VByte	17
9.3	Влияние сжатия на размер и скорость (качественное описание)	17
9.4	Юнит-тестирование	17

1 Постановка задачи и ограничения

1.1 Общая цель

Целью работы является разработка компонентов поисковой системы на базе собственного корпуса документов единой тематики (мировой транспорт и инфраструктура), включая:

- добычу корпуса и анализ структуры документов;
- поискового робота (обкачка/переобкачка);
- токенизацию и статистику;
- анализ распределения частот терминов и сравнение с законом Ципфа;
- построение булева индекса и реализацию булевого поиска.

1.2 Ограничения реализации

Основные компоненты поисковой системы должны быть реализованы на C/C++ без использования контейнеров STL. Исключение: STL допускается применять для токенизации (например, `std::string`, `std::vector` внутри токенизатора). Компоненты обвязки и выкачки разрешено реализовывать на интерпретируемых языках (Python) и утилитами командной строки.

Кодировка ввода-вывода единая: UTF-8.

Текущее размещение на диске (пример):

- `C:\Users\user\YandexDisk\corpus_aviation\` — сырые данные корпуса;
- `C:\Users\user\YandexDisk\MAI_IR\` — рабочая папка проекта (код и производные файлы).

2 Лабораторная 1. Добыча корпуса документов

2.1 Источник данных и тематика

В качестве источников данных использовались открытые энциклопедические ресурсы, предоставляющие доступ к полным текстам статей.

Основным источником является русскоязычная Википедия. Дополнительным источником выступает специализированная авиационная энциклопедия SKYbrary, посвящённая вопросам эксплуатации воздушных судов, авиационной безопасности и инфраструктуры гражданской авиации.

Тематика формируемого корпуса охватывает мировой транспорт и транспортную инфраструктуру, включая авиацию, железнодорожный транспорт, морские и речные порты, дорожную сеть, мосты, аэропорты и связанные с ними объекты.

Для формирования корпуса применяется обход категорий и подкатегорий источников с последующим извлечением текстов статей.

2.2 Формат сырых документов

Сырые документы сохраняются в формате JSONL (одна строка соответствует одному документу корпуса).

Минимальный набор полей документа:

- **source** — источник документа (Wikipedia, SKYbrary);
- **pageid** или **url** — идентификатор статьи (числовой идентификатор для Википедии либо URL для SKYbrary);
- **title** — заголовок статьи;
- **text** — извлечённый текст статьи в формате plain-text.

Использование формата JSONL позволяет обрабатывать корпус построчно и обеспечивает удобное продолжение загрузки в случае прерывания процесса.

2.3 Выделение текста

Для дальнейшей индексации используется исключительно текстовое содержимое статей. Из сырых документов извлекается поле **text**, содержащее очищенный текст без HTML-разметки, навигационных элементов и служебной информации.

HTML-разметка и вспомогательные элементы страниц не сохраняются в итоговом корпусе, поскольку они относятся к задаче веб-робота и не используются на этапе построения поискового индекса.

2.4 Журнал выполнения и типовые проблемы

- Ошибки 403 при обращении к API источников — устранены добавлением корректного заголовка User-Agent и соблюдением интервалов между запросами.
- Ошибки, связанные с ограничением дискового пространства — корпус размещён в папке Яндекс.Диска; выполнена очистка временных файлов.
- Сетевые ошибки (getaddrinfo failed, ConnectionResetError) — реализована возможность продолжения загрузки с места остановки за счёт батчевой записи документов.

2.5 Пример выполнения программы

На рисунке 1 приведён фрагмент консольного вывода программы, демонстрирующий загрузку списка страниц корпуса и продолжение обработки документов с учётом ранее сохранённых батчей.

```
[INFO] Загружен список страниц из C:\Users\user\YandexDisk\corpus_aviation\pages_list.json, всего: 112920
[INFO] Загружен список страниц из C:\Users\user\YandexDisk\corpus_aviation\pages_list.json, всего: 112920
[DL] Уже обработано документов (по существующим батчам): 79289
```

Рис. 1: Фрагмент журнала выполнения программы при загрузке и обработке корпуса документов

3 Лабораторная 2. Поисковый робот

3.1 Требования

Поисковый робот принимает единственным аргументом путь к YAML-конфигурации. Конфиг содержит:

- секция **db**: параметры базы данных;

- секция `logic`: задержки и логика обкачки;
- дополнительные параметры (например, лимиты, источники, политика повторов).

В базу данных сохраняются поля:

- URL (нормализованный);
- сырой HTML;
- название источника;
- дата обкачки (Unix timestamp).

Робот должен:

- корректно останавливаться и продолжать с места остановки;
- периодически переобкачивать страницы и сохранять новую версию только при изменениях.

3.2 Метод решения

Архитектурно робот организован как:

- очередь URL на обработку;
- хранилище состояния в базе (`processed`, `etag/last-modified`, время последней обкачки);
- HTTP-запросы с условной загрузкой (`If-None-Match` / `If-Modified-Since`);
- обработка ошибок сети с повторами и экспоненциальной задержкой.

3.3 Схема хранения документов

В MongoDB каждый документ хранится со следующими полями:

- `url` — нормализованный URL;
- `html` — исходный HTML-текст страницы;
- `source` — строковый идентификатор источника;
- `fetches_at` — время обкачки (Unix time stamp);

3.4 Ключевые фрагменты реализации

Чтение YAML-конфигурации

```
with open(config_path, "r", encoding="utf-8") as f:
    cfg = yaml.safe_load(f)
db_cfg = cfg["db"]
logic_cfg = cfg["logic"]
```

Уникальность по URL в базе

```
self.col.create_index([("url", ASCENDING)], unique=True)
```

Условная загрузка и переобкачка

```
headers = {}
if doc.get("etag"): headers["If-None-Match"] = doc["etag"]
if doc.get("last_modified"): headers["If-Modified-Since"] = doc["last_modified"]

r = session.get(url, headers=headers, timeout=30)
if r.status_code == 304:
    #
    return
```

Сохранение документа

```
record = {
    "url": norm_url,
    "html": html,
    "source": source_name,
    "fetched_at": int(time.time()),
    "etag": r.headers.get("ETag"),
    "last_modified": r.headers.get("Last-Modified"),
}
self.col.update_one({"url": norm_url}, {"$set": record}, upsert=True)
```

3.5 Экспериментальная проверка работы поискового робота

Для проверки корректности работы поискового робота был выполнен запуск на корпусе, содержащем более 110 000 страниц русскоязычной Википедии по тематике транспорта. Начальный список URL был загружен из файла `pages_list.json`. В процессе работы робот последовательно обкачивал страницы, сохраняя исходный HTML-код документов в базе данных MongoDB.

Ниже приведён фрагмент консольного вывода, демонстрирующий процесс импорта начальных URL, загрузку документов и корректную обработку страниц.

```
1 [INIT] Crawler initialized
2 [SEEDS] Loading pages list from C:/Users/user/YandexDisk/corpus_aviation/pages_list.json
3 [SEEDS] Seeds imported (processed=112920), documents in DB: 112920
4 [MAIN] Crawling: https://ru.wikipedia.org/wiki/
5 [MAIN] Downloaded, size 267.0 KB, changed=True
6 [MAIN] Crawling: https://ru.wikipedia.org/wiki/
7 [MAIN] Downloaded, size 82.1 KB, changed=True
8 [MAIN] Crawling: https://ru.wikipedia.org/wiki/_
9 [MAIN] Downloaded, size 116.0 KB, changed=True
10 [MAIN] Crawling: https://ru.wikipedia.org/wiki/_
11 [MAIN] Downloaded, size 93.1 KB, changed=True
```

Из приведённого вывода видно, что поисковый робот корректно загрузил начальный список URL, содержащий 112 920 страниц, после чего начал последовательную обкачку документов. Для каждой страницы выводится информация о размере загруженного HTML-документа, что позволяет оценить объём передаваемых данных. Флаг `changed=True` указывает на то, что документ был сохранён или обновлён в базе данных.

4 Токенизация

На данном этапе был реализован процесс токенизации текстового корпуса, используемый в дальнейшем при построении поискового индекса. Под токенизацией понимается разбиение

входного текста на последовательность элементарных лексических единиц — токенов.

4.1 Метод токенизации

Токенизация реализована в виде отдельной утилиты на языке C++. Алгоритм основан на последовательном проходе по символам входного текста и накоплении непрерывных последовательностей допустимых символов.

Ключевая функция определения допустимых символов токена приведена ниже:

```
bool is_word_char(unsigned char c) {
    if (c >= '0' && c <= '9') return true;
    if (c >= 'A' && c <= 'Z') return true;
    if (c >= 'a' && c <= 'z') return true;
    if (c == '_' || c == '-') return true;
    if (c >= 0xC0) return true;
    return false;
}
```

Таким образом, в токены включаются буквы латинского и кириллического алфавитов, цифры, а также символы подчёркивания и дефиса. Все прочие символы используются в качестве разделителей.

Основной цикл токенизации реализован следующим образом:

```
for (size_t i = 0; i < text.size(); ++i) {
    unsigned char c = static_cast<unsigned char>(text[i]);
    if (is_word_char(c)) {
        cur.push_back(std::tolower(c));
    } else {
        if (!cur.empty()) {
            tokens.push_back(cur);
            cur.clear();
        }
    }
}
```

Каждый выделенный токен приводится к нижнему регистру, что позволяет унифицировать представление терминов при последующей индексации.

Пример результата токенизации фрагмента текста корпуса:

```
быстрый
парусный
тендер
или
катер
а
затем
ластовое
судно
черноморского
флота
российской
```


империи
находившееся
в
составе
флота
с
1833

5 Лемматизация

Для повышения качества поиска и анализа распределения терминов была реализована лемматизация — приведение словоформ к их нормальной (словарной) форме.

5.1 Метод лемматизации

Лемматизация реализована в виде отдельного этапа анализа корпуса с использованием морфологического анализатора `py morphology2`. Данный этап выполнен на языке Python и применяется поверх уже выполненной токенизации.

Функция определения леммы для одного токена реализована следующим образом:

```
def lemmatize_token(tok):  
    if any("а" <= ch <= "я" or ch == "ё" for ch in tok):  
        p = morph.parse(tok)[0]  
        return p.normal_form  
    else:  
        return tok
```

Лемматизация применяется только к токенам, содержащим символы кириллического алфавита. Токены, не относящиеся к русскому языку (например, числа или латиница), сохраняются без изменений.

Основной цикл обработки корпуса выглядит следующим образом:

```
for tok in tokenize(text):  
    base_freqs[tok] = base_freqs.get(tok, 0) + 1  
    lemma = lemmatize_token(tok)  
    lemma_freqs[lemma] = lemma_freqs.get(lemma, 0) + 1
```

Таким образом, одновременно накапливаются частоты исходных токенов и частоты лемм.

5.2 Сравнение частот до и после лемматизации

Фрагмент распределения частот без лемматизации:

в	12630
и	7413
на	5638
года	3433
был	1445
году	1349
была	886
станции	942
станция	770

После применения лемматизации соответствующие словоформы объединяются, например:

- года, году → год;
- был, была → быть;
- станции, станция → станция.

Это приводит к уменьшению количества уникальных терминов и перераспределению частот в пользу семантически значимых лемм.

5.3 Результаты после лемматизации

После применения лемматизации наблюдается перераспределение частот терминов, связанное с объединением различных словоформ в единую нормальную форму. Это приводит к уменьшению количества уникальных терминов и росту частот наиболее значимых лемм.

Фрагмент распределения частот после лемматизации:

в	13202
и	7413
год	6044
на	5638
с	4375
быть	3913
корабль	2334
станция	2057
флот	1626

По сравнению с распределением без лемматизации, словоформы года, году были объединены в лемму год, а формы был, была — в лемму быть. Аналогичное объединение произошло для терминов станция и флот.

Таким образом, лемматизация позволила сконцентрировать частоты на семантически значимых терминах, что потенциально повышает полноту поиска. При этом высокочастотные служебные слова (например, в, и, не) сохраняются, так как их фильтрация не является задачей данного этапа.

5.4 Анализ влияния на качество поиска

Лемматизация повышает полноту поиска, позволяя находить документы, содержащие различные словоформы одного и того же слова. В то же время возможно ухудшение качества поиска в отдельных случаях, связанных с омонимией и потерей грамматического контекста, что является известным ограничением данного подхода.

5.5 Закон Ципфа

Для построенного корпуса документов была проведена проверка выполнения закона Ципфа. Анализ проводился дважды:

- для токенов без лемматизации;
- для лемматизированных терминов.

Для каждого случая строилось распределение частот терминов по убыванию, после чего данные отображались в логарифмических координатах: $\log(\text{rank})$ и $\log(\text{freq})$. Аппроксимация закона Ципфа выполнялась методом наименьших квадратов по первым 1000 наиболее частотным терминам.

5.5.1 Распределение токенов без лемматизации

На рис. 2 представлено распределение частот токенов без предварительной лемматизации.

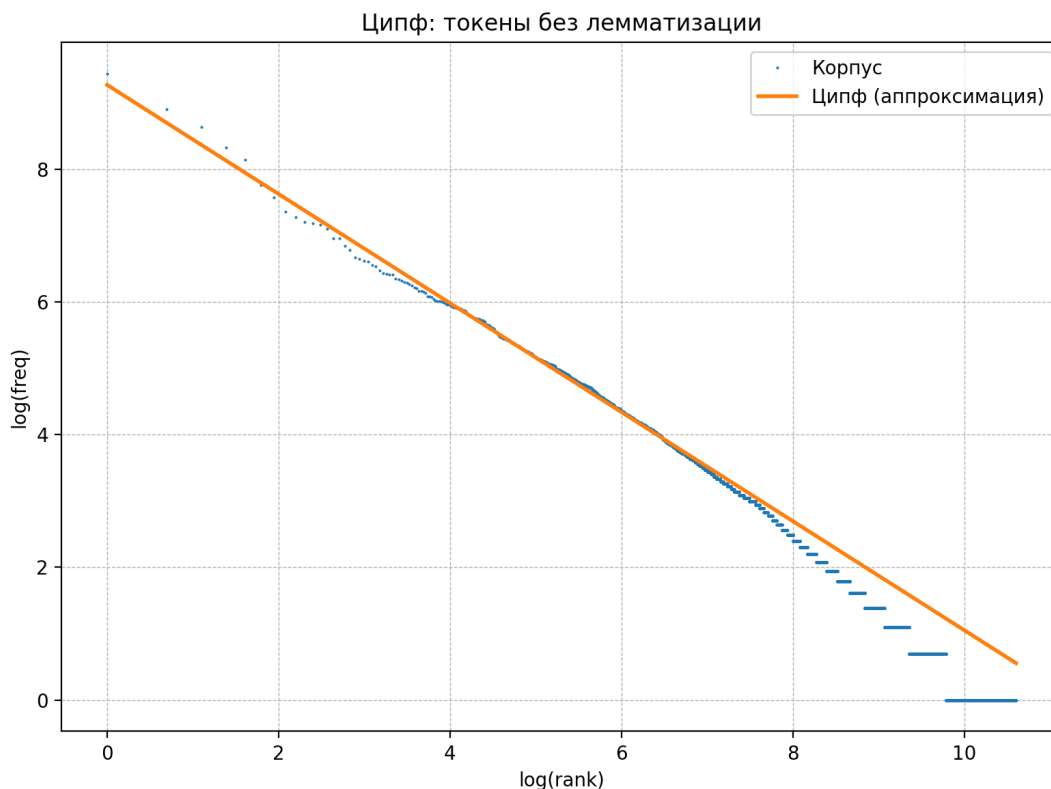


Рис. 2: Закон Ципфа для токенов без лемматизации

Вывод программы при аппроксимации закона Ципфа:

Оценка Ципфа: $\log(C)=9.274387$, $s=0.822159$, $\text{fit_points}=1000$

Полученное распределение близко к линейному в логарифмических координатах, что соответствует классическому закону Ципфа. Незначительные отклонения наблюдаются:

- в области малых рангов — из-за доминирования служебных слов;
- в хвосте распределения — из-за большого количества редких терминов.

В целом, распределение токенов без нормализации хорошо описывается степенным законом.

5.5.2 Распределение после лемматизации

На рис. 3 приведено распределение частот терминов после применения лемматизации.

Вывод программы для лемматизированного корпуса:

Оценка Ципфа: $\log(C)=9.674809$, $s=0.854012$, $\text{fit_points}=1000$

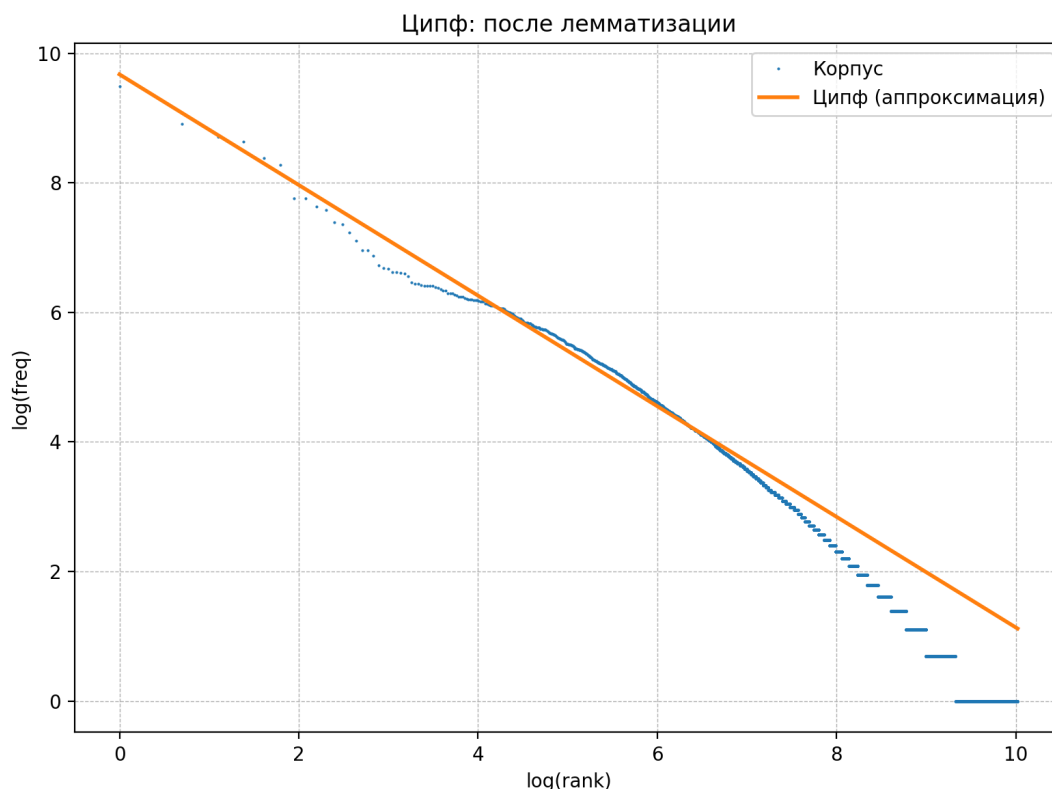


Рис. 3: Закон Ципфа после лемматизации

После лемматизации наблюдается более заметное отклонение распределения от классического закона Ципфа. Это связано с тем, что лемматизация агрегирует различные словоформы в одну лемму, например:

- год, года, году $\rightarrow$ год;
- был, была, были $\rightarrow$ быть.

В результате:

- резко возрастают частоты наиболее популярных терминов;
- сокращается длина хвоста распределения;
- линейная аппроксимация в логарифмических координатах становится менее точной.

5.5.3 Сравнение и выводы

Сравнение двух распределений показывает, что:

- закон Ципфа лучше выполняется для сырых токенов без лемматизации;
- лемматизация приводит к систематическому искажению распределения частот;
- увеличение параметра s после лемматизации является ожидаемым эффектом агрегации словоформ.

Таким образом, ухудшение соответствия закону Ципфа после лемматизации не является ошибкой реализации, а отражает изменение статистических свойств корпуса. При этом лемматизация остаётся полезной для задач информационного поиска, так как снижает размер словаря и повышает полноту поиска, несмотря на ухудшение соответствия степенному распределению.

6 Токенизация и нормализация текста

6.1 Цель и общая схема

Цель токенизации — преобразовать входной текст документа (в кодировке UTF-8) в последовательность термов (токенов), пригодных для построения булевого индекса и дальнейшего поиска. Дополнительно для каждого токена фиксируется позиция появления в документе (порядковый номер токена), что позволяет хранить координатную информацию.

Алгоритм состоит из двух этапов:

1. **Нормализация символов** (приведение регистра, обработка кириллицы и опциональная свёртка “ё” → “е”).
2. **Выделение токенов** по правилам допустимых символов токена и правилам обработки соединителей (дефис/подчёркивание).

6.2 Входные и выходные данные

Вход: строка `text` длиной `text_len` байт, содержащая UTF-8.

Выход: структура `TokenList`, содержащая массив токенов:

- `Token.s` — C-строка (нулевой терминатор) с байтовым содержимым токена в UTF-8;
- `Token.len` — длина токена в байтах (без завершающего ‘\0’);
- `Token.pos` — позиция токена в документе (счётчик токенов, начиная с 0).

6.3 Настройки токенизации

Поведение задаётся структурой `TokenizerConfig`:

- `max_token_bytes` — максимальная длина токена в байтах; более длинные токены усекаются;
- `fold_yo_to_e` — если включено, выполняется свёртка “ё” → “е” (и “Ё” → “Е” после приведения регистра);
- `keep_joiners` — если включено, дефис - и подчёркивание _ могут сохраняться внутри токена при соблюдении условий.

6.4 Декодирование UTF-8 и представление символов

Текст обрабатывается посимвольно, но физически хранится в UTF-8 (переменная длина кодовой единицы). Для каждого шага алгоритм выполняет декодирование одного Unicode code point:

- 1 байт: 0xxxxxxx
- 2 байта: 110xxxxx 10xxxxxx
- 3 байта: 1110xxxx 10xxxxxx 10xxxxxx
- 4 байта: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Если последовательность байт некорректна (ошибка UTF-8, обрыв на середине символа, запрещённые суррогаты и т.п.), текущий байт трактуется как разделитель: текущий токен сбрасывается, а указатель входа продвигается на 1 байт.

6.5 Нормализация символов

Для каждого декодированного code point выполняется нормализация:

1. Приведение латиницы к нижнему регистру: 'A'... 'Z' → 'a'... 'z'.
2. Приведение русской кириллицы к нижнему регистру: U+0410...U+042F → U+0430...U+044F.
3. Приведение U+0401 ("Ё") к U+0451 ("ё").
4. Если `fold_yo_to_e = 1`, то U+0451 ("ё") → U+0435 ("е"), а U+0401 ("Ё") → U+0415 ("Е") после приведения регистра.

Нормализованный code point кодируется обратно в UTF-8 и добавляется в буфер текущего токена.

6.6 Правила формирования токенов

Токен формируется как последовательность "токенных" символов. Символ считается допустимым, если он принадлежит одному из классов:

- ASCII-цифры: '0'... '9';
- ASCII-латиница: 'a'... 'z' (после нормализации);
- кириллица: U+0400...U+052F (после нормализации);
- соединители - и _ (при включённом `keep_joiners` и выполнении условий ниже).

Соединители - и _ сохраняются *только внутри токена*: если текущий токен непустой и следующий символ (после соединителя) является буквой/цифрой (латиница или кириллица), то соединитель добавляется в токен. В противном случае соединитель рассматривается как разделитель и вызывает сброс текущего токена.

При завершении токена выполняется `trimming`: с начала и конца удаляются - и _. Если после `trimming` токен пуст, он отбрасывается.

6.7 Позиции токенов

Позиция `pos` — это порядковый номер добавленного токена. Счётчик увеличивается на 1 для каждого успешно добавленного токена (после `trimming` и усечения по `max_token_bytes`). Таким образом, позиции соответствуют последовательности термов, а не байтовым сдвигам.

6.8 Ограничения по длине

Размер буфера токена ограничен `max_token_bytes`. Если добавление очередного символа приводит к превышению лимита, дополнительные байты не добавляются (токен остаётся усечённым). Это ограничивает потребление памяти и предотвращает создание чрезмерно длинных термов.

6.9 Результат токенизации

Результатом является массив токенов `TokenList`, где каждый элемент содержит: (`term`, `len`, `pos`). Этот список используется:

- при индексации — для накопления частот и координат в рамках документа;
- при обработке запроса — для нормализации термов и приведения их к формату словаря индекса.

7 Булев индекс

7.1 Токенизация и нормализация

Внутреннее представление документа после токенизации:

- документ рассматривается как последовательность токенов;
- для каждого терма сохраняется список координат (позиции токена в документе, начиная с 0);
- нормализация: понижение регистра (латиница и кириллица); токен-символами являются буквы и цифры (а также, при необходимости, дефис/подчёркивание).

7.2 Бинарный формат хранения

7.2.1 `direct.bin` (прямой индекс)

Побайтовая схема:

- `u32 doc_count`
- для каждого документа $i = 1..doc_count$:
 - `u32 title_len`, затем `title_len` байт UTF-8 заголовка
 - `u32 url_len`, затем `url_len` байт UTF-8 URL

Идентификатор документа в постингах: **docId** начинается с 1.

7.2.2 `vocab.bin` (словарь)

Побайтовая схема:

- `u32 terms_count`
- для каждого терма в лексикографическом порядке:
 - `u32 term_len`, затем `term_len` байт UTF-8 терма
 - `u64 offset` — смещение в `postings.bin`
 - `u32 df` — document frequency (количество документов, где встречается терм)

7.2.3 postings.bin (обратный индекс)

Для каждого термина по смещению `offset` хранится блок:

- `df` (VByte)
- затем для каждого из df документов:
 - `doc_gap` (VByte), где `doc = prev_doc + doc_gap`
 - `tf` (VByte) — число вхождений термина в документе
 - затем tf раз: `pos_gap` (VByte), где `pos = prev_pos + pos_gap`

Таким образом, координатный блок — это последовательность закодированных `pos_gap`.

7.3 Метод сортировки

Словарь формируется в памяти и затем сортируется стандартной `qsort` по `strcmp`. Достоинства: простота реализации, приемлемо для умеренных корпусов. Недостатки: зависимость от объема RAM (индекс строится “в памяти”), отсутствует внешний (external) sort; при росте данных возможна деградация из-за аллокаций и промахов кэша.

8 Булев поиск и ранжирование (ЛР4 + ЛР8)

8.1 Синтаксис запросов и парсер

Реализованы операторы:

- пробел или `&&` — логическое И,
- `||` — логическая операция ИЛИ,
- `!` — логическое НЕТ,
- скобки ().

Парсер строит дерево выражения и вычисляет его над битсетами документов.

8.2 Модель выполнения запроса

1. для термина считывается список документов из `postings.bin`, формируется битсет присутствия;
2. для операций AND/OR/NOT выполняются побитовые операции над битсетами;
3. множество документов результата преобразуется в список `docId`;
4. документы сортируются по TF-IDF.

8.3 TF–IDF ранжирование

Для каждого термина запроса извлекаются (doc, tf) из постингов. Вес термина:

$$idf(t) = \log \frac{N + 1}{df(t) + 1} + 1$$

$$tfw(t, d) = 1 + \log(tf(t, d))$$

Итоговый скор документа:

$$score(d) = \sum_{t \in Q} tfw(t, d) \cdot idf(t)$$

Сортировка выдачи выполняется по $score(d)$ по убыванию.

8.4 Примеры запросов (удачные и неудачные)

Примеры удачной выдачи (фрагмент вывода)

9742 Robinson R22 - лёгкий коммерческий двухместный вертолёт производства
19746 Sikorsky CH-53 Super Stallion - тяжёлый вертолёт, эксплуатируемый ...

Примеры запросов для демонстрации

- Булев: самолёт && вертолёт — строгая фильтрация, затем TF–IDF сортировка.
- Булев со скобками: самолёт && (двигатель || шасси).
- Нечёткий (по ТЗ ЛР8): московский авиационный институт.
- Неудачный: запрос с термом, которого нет в словаре.

8.5 Тестирование корректности (как обосновано)

Корректность проверялась:

- модульными тестами на синтетическом корпусе (фиксированный набор документов и ожидаемые docId для выражений AND/OR/NOT/скобок);
- ручными проверками на корпусе: выбор термина и контроль, что документы из выдачи действительно содержат его;
- проверкой граничных случаев: неизвестные термины, пустой результат, лишние пробелы, разные регистры.

9 Координатное сжатие (ЛР6)

9.1 Выбранный метод сжатия и причины выбора

Используется комбинация:

- **gap-encoding** (дельты) для docId и позиций: doc_gap, pos_gap;
- **Variable Byte** (VByte) для кодирования целых чисел.

Причины выбора: простота, быстрый декодер, хорошее сжатие для малых значений (а дельты обычно малы), широко применим для постингов и координатных блоков.

9.2 Побитовая схема VByte

Каждый байт:

- старший бит (bit7) — **признак продолжения**: 1 означает “есть следующий байт”, 0 означает “последний байт числа”;
- младшие 7 бит (bit0..bit6) — данные числа.

Число хранится в базе 128, “младшие группы” по 7 бит записываются первыми.

9.3 Влияние сжатия на размер и скорость (качественное описание)

Сжатие уменьшает объём координатных блоков за счёт того, что кодируются не абсолютные значения, а дельты, которые, как правило, малы и хорошо упаковываются VByte. При этом поиск остаётся корректным, так как при чтении выполняется последовательное восстановление исходных docId и позиций путём накопления doc_gap и pos_gap.

Как обосновывается корректность поиска после сжатия. Сжатие не меняет смысла данных: сохраняются те же последовательности docId и позиций, только в виде gap-значений, кодированных VByte. Декодирование выполняется строго последовательно:

- docId восстанавливается суммированием doc_gap;
- позиции восстанавливаются суммированием pos_gap;
- порядок и количество элементов совпадают с исходными значениями.

Корректность подтверждается модульными тестами (сравнение ожидаемых docId для выражений) и ручной валидацией на корпусе.

9.4 Юнит-тестирование

Для проверки корректности реализации ключевых компонентов использованы автоматические юнит-тесты двух типов: (1) нативные тесты на C/C++ для проверок бинарей индексатора/поисковика и модуля токенизации; (2) тесты на Python (pytest) для модуля краулера, изолированные от сети и реальной базы данных за счёт моков.

Тесты индексатора и поисковика (C++ test_runner.cpp). Тестовый раннер создаёт временную рабочую директорию test_tmp, генерирует небольшой корпус из 5 документов и запускает внешние исполняемые файлы indexer и searcher как отдельные процессы. Вывод программ перехватывается (на Windows через CreateProcess и пайп, на POSIX через popen), после чего автоматически проверяется код завершения и содержимое результатов.

Покрываемые сценарии:

- **Построение индекса:** проверяется успешное завершение индексатора и наличие в stdout строки indexed docs: 5.
- **Булева конъюнкция (&&):** запрос эсминец && конвой должен вернуть ровно документ #1.

- **Булева дизъюнкция (||) и ранжирование:** запрос `эсминец || конвой` должен вернуть документы #1, #2, #4, #5 в порядке TF-IDF (проверяется именно порядок id).
- **Отрицание (!):** запрос `!конвой` должен вернуть #3 и #4.
- **Скобки и приоритеты:** запрос `эсминец && (конвой || фрегат)` должен вернуть #1 и #4.
- **«Нечёткий» режим без операторов:** запрос `эсминец конвой` (без `&&`, `||`, `!`) должен вести себя как дизъюнкция с ранжированием и вернуть #1, #2, #4, #5.

Результаты поисковика извлекаются из `stdout` построчно: из каждой строки берётся `docid` (до символа табуляции либо вся строка целиком) и сравнивается со строго ожидаемым списком. При несовпадении тест немедленно завершает выполнение с диагностикой ожидаемых и полученных id.

Тесты токенизатора (C, unit для tokenizer). Юнит-тесты токенизатора вызывают функцию `tokenize_utf8_with_positions` и сравнивают полученный список токенов с эталоном: последовательность нормализованных термов и их позицию `pos` (порядковый номер терма в документе). Для отдельных проверок тестируется также `normalize_term_utf8`.
Покрываемые сценарии:

- **Нормализация регистра ASCII:** `"Hello, WORLD!"` → `hello, world`.
- **Нормализация регистра кириллицы:** `"ЭсМинЕц конвой"` → `эсминец, конвой`.
- **Оptionальная свёртка ё в е:** при `fold_yo_to_e=1` строки вида `"самолёт"` и `"самолет"` приводятся к одному терму.
- **Режимы обработки соединителей:** при `keep_joiners=1` сохраняются токены вида `ан-12`, `ch-53`, `a_b`; при `keep_joiners=0` соединители считаются разделителями и токены дробятся (`Ан-12` → `ан, 12`).
- **Позиции токенов:** проверяется корректное увеличение `pos` на простом примере `"a b a"`.
- **Ограничение длины токена:** при `max_token_bytes=16` длинный терм усекается до 16 байт.
- **Некорректный UTF-8 как разделитель:** байт `0xFF` в потоке трактуется как граница токенов (`"a <0xFF> b"` → `a, b`).
- **Нормализация терма целиком:** `"ЁЖИК-12"` → `"ежик-12"` при включённой свёртке ё в е.

Тесты краулера (Python, pytest). Для краулера применён `pytest` с изоляцией внешних зависимостей. Реальная MongoDB заменена фейковой коллекцией `FakeCollection` (хранит документы в памяти и реализует минимальный набор методов: `estimated_document_count`, `update_one`, `find_one`). Создание клиента Mongo патчится через `unittest.mock.patch`, задержки сна отключаются, сетевые запросы мокируются через подмену `session.get`.

Покрываемые сценарии:

- **Загрузка сидов:** при пустой коллекции `ensure_seeds_loaded` должен добавить 1 документ со статусом `new` и корректным URL на основе `base_url`.

- **Пропуск загрузки сидов при непустой базе:** если коллекция уже содержит документы, повторная загрузка не должна добавлять новые записи.
- **Выбор следующего документа:** `_pick_next_document` должен предпочитать документ со статусом `new` над `done`, если `done` ещё не подошёл по `next_crawl_at`.
- **Обработка сетевых ошибок:** исключение при `get` приводит к результату `status=error` и заполнению поля `error`.
- **HTTP 304 Not Modified:** корректно возвращается `status=not_changed`.
- **HTTP $\neq$ 200:** например, 500 приводит к `status=error` и сообщению HTTP 500.
- **HTTP 200 OK:** сохраняются `html`, а также метаданные `ETag` и `Last-Modified`.
- **Полный проход цикла обработки:** `run()` должен обработать один документ, выставить статус `done`, сохранить HTML и вычислить `content_hash`.

Критерий успешности. Все тестовые наборы завершаются выводом `ALL TESTS PASSED` и кодом возврата 0. Любое расхождение фактического результата с ожидаемым (код завершения процесса, количество документов, список `docid`, нормализация термов, статусы краулера) приводит к немедленному падению теста с диагностическим сообщением.